

BODA

Trusted Rides & Delivery — Gulu, Uganda

Full Technical Architecture & Developer Documentation

MVP Focus
Rides + Delivery

Launch City
Gulu, Uganda

Payments
MTN + Airtel MoMo

Version 1.0 | Confidential | 16 June 2026

1. Product Overview

Boda is a trust-infrastructure platform layered on top of Uganda's existing boda boda network. The platform adds verified rider identity, mobile money escrow payments, real-time GPS tracking, and community reputation ratings to an informal transport ecosystem that currently has none of these. Think Uber — but built for northern Uganda, starting in Gulu.

1.1 Problem Statement

Boda boda riders in Gulu are informal, unverifiable, and unaccountable. Customers face:

- No way to verify a rider's identity or trustworthiness before hiring them
- No recourse when goods are not delivered or rides go wrong
- No payment trail — everything is cash with zero accountability
- No rating or reputation system to distinguish good riders from bad

1.2 Solution

Boda provides:

- Verified rider profiles with National ID, plate number, and photo confirmation
- Mobile money escrow — customers pay upfront, riders receive only on confirmed delivery
- Live GPS tracking for both rides and deliveries
- Community ratings that build rider reputation over time
- Admin dashboard for ongoing rider verification and dispute resolution

1.3 MVP Scope

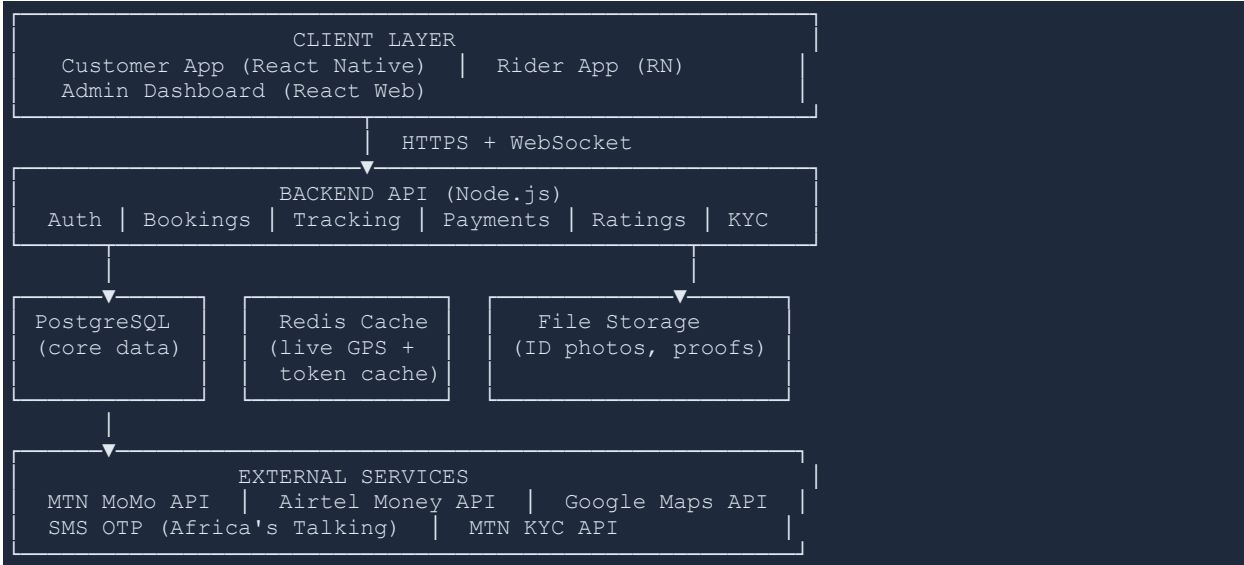
MVP Rule: Build only what validates the core trust proposition. Do not over-engineer. Add features after launch when real users tell you what they need.

Customer App	Rider App	Admin Dashboard
Phone OTP signup	Register with National ID	Review rider applications
Request ride or delivery	Accept / decline requests	Verify / suspend riders
See nearby verified riders	Navigate to pickup & dropoff	Monitor live bookings
Live tracking of rider	Receive MTN/Airtel payment	Handle disputes
Pay via MTN/Airtel/Cash	Build rating & reputation	View transaction logs
Rate rider after trip		

OUT OF SCOPE (MVP): Scheduled bookings, multi-stop trips, in-app chat, surge pricing, promotions, referral system. Add after launch.

2. System Architecture

2.1 High-Level Architecture



2.2 Tech Stack

Layer	Technology	Rationale
Backend	Node.js + Fastify	Non-blocking I/O ideal for real-time ride matching. Handles thousands of concurrent connections.
Database	PostgreSQL	ACID-compliant for financial data. Handles millions of records. Scales with read replicas.
Real-time	Socket.io + Redis adapter	Live GPS tracking. Redis adapter allows multi-server scaling later.
Mobile	React Native	Single codebase for Android + iOS. Android first — higher penetration in Uganda.
Admin UI	React.js + Tailwind	Fast to build. Rider verification dashboard.
Payments	MTN MoMo + Airtel Money APIs	Uganda-native mobile money. No bank account required by users.
Maps	Google Maps API	Best GPS coverage in Gulu. Navigation for riders.
Auth	Phone OTP + JWT	Most Ugandans use phone numbers, not email.

OTP/SMS	Africa's Talking API	Uganda SMS gateway. Reliable, affordable, local support.
File Storage	AWS S3 / Cloudflare R2	Rider ID photos, delivery proof photos.
Hosting	Railway.app (MVP) → AWS (scale)	Zero DevOps to start. Migrate when traffic demands it.
Cache	Redis	Token caching, live driver locations, session state.

Scalability Note: This stack supports Gulu MVP through all-of-Uganda and into East Africa without fundamental changes. The only scaling additions needed at large scale are: Redis Socket.io adapter (already included), PostgreSQL read replicas, and migration from Railway to AWS ECS or Kubernetes.

3. Database Schema

3.1 Core Tables

```
-- =====
-- USERS (customers)
-- =====

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone       VARCHAR(15) UNIQUE NOT NULL, -- e.g. 256771234567
  name        VARCHAR(100),
  profile_photo TEXT,                      -- S3 URL
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- =====
-- RIDERS
-- =====

CREATE TABLE riders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  national_id VARCHAR(20) UNIQUE NOT NULL,
  plate_number VARCHAR(10) NOT NULL,
  id_photo    TEXT,                      -- S3 URL
  selfie_photo TEXT,                    -- S3 URL
  status      VARCHAR(20) DEFAULT 'pending',
              -- pending | verified | suspended | rejected
  current_lat DECIMAL(10, 8),
  current_lng DECIMAL(11, 8),
  is_online   BOOLEAN DEFAULT false,
  avg_rating  DECIMAL(3,2) DEFAULT 0.00,
  total_trips INTEGER DEFAULT 0,
  created_at  TIMESTAMP DEFAULT NOW()
);
```

```
-- =====
-- BOOKINGS
-- =====

CREATE TABLE bookings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID REFERENCES users(id),
  rider_id    UUID REFERENCES riders(id),
  type        VARCHAR(10) NOT NULL, -- ride | delivery
  pickup_lat  DECIMAL(10, 8) NOT NULL,
  pickup_lng  DECIMAL(11, 8) NOT NULL,
  pickup_address TEXT,
  dropoff_lat DECIMAL(10, 8) NOT NULL,
  dropoff_lng DECIMAL(11, 8) NOT NULL,
  dropoff_address TEXT,
  status      VARCHAR(20) DEFAULT 'pending',
              -- pending | accepted | in_progress
              -- | completed | cancelled
  fare_estimate INTEGER,                -- UGX (no decimals)
  fare_final   INTEGER,
  distance_km  DECIMAL(6,2),
  created_at  TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP
);
```

```

-- =====
-- DELIVERIES (linked to bookings)
-- =====
CREATE TABLE deliveries (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id        UUID REFERENCES bookings(id),
  item_description  TEXT NOT NULL,
  recipient_name    VARCHAR(100),
  recipient_phone    VARCHAR(15),
  confirmation_code  VARCHAR(6),          -- 6-digit OTP for delivery confirm
  photo_proof       TEXT,                 -- S3 URL of delivery photo
  confirmed_at      TIMESTAMP
);

```

```

-- =====
-- RATINGS
-- =====
CREATE TABLE ratings (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id        UUID REFERENCES bookings(id),
  rated_by          UUID REFERENCES users(id),
  rider_id          UUID REFERENCES riders(id),
  score             SMALLINT CHECK (score BETWEEN 1 AND 5),
  comment           TEXT,
  created_at        TIMESTAMP DEFAULT NOW()
);

-- =====
-- PAYMENTS
-- =====
CREATE TABLE payments (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id        UUID REFERENCES bookings(id),
  amount            INTEGER NOT NULL,      -- UGX
  method            VARCHAR(10),           -- mtn | airtel | cash
  status            VARCHAR(20) DEFAULT 'pending',
                                     -- pending | held | released | refunded | failed
  transaction_ref    VARCHAR(100),         -- MoMo reference ID
  held_at           TIMESTAMP,
  released_at       TIMESTAMP,
  created_at        TIMESTAMP DEFAULT NOW()
);

```

4. Payment Integration

4.1 Overview

Both MTN Mobile Money and Airtel Money expose open REST APIs for Uganda. Both have developer sandboxes you can test against before going live. You need two operations from each:

- Collections — charging a customer for a ride or delivery
- Disbursements — paying a rider their earnings after completion

	MTN MoMo	Airtel Money
Developer Portal	momodeveloper.mtn.com	developers.airtel.africa
Auth Method	Basic Auth → Bearer token	Client credentials → Bearer token
Token Expiry	1 hour	1 hour
Collections	/collection/v1_0/requesttopay	/merchant/v1/payments/
Disbursements	/disbursement/v1_0/transfer	/standard/v1/disbursements/
Status Check	GET with reference ID	GET with transaction ID
Currency	UGX	UGX
Sandbox	Free, available immediately	Free, available immediately
Production	Requires MTN merchant agreement	Requires Airtel AMCUL agreement

4.2 Network Detection

Detect MTN vs Airtel from the phone number prefix before calling either API:

```
// services/paymentService.js

const detectNetwork = (phoneNumber) => {
  // Uganda phone format: 256XXXXXXXXX
  // MTN prefixes: 076, 077, 078, 039
  // Airtel prefixes: 070, 075, 074
  const prefix = phoneNumber.slice(3, 5); // after country code 256
  const mtnPrefixes = ['76', '77', '78', '39'];
  const airtelPrefixes = ['70', '75', '74'];

  if (mtnPrefixes.includes(prefix)) return 'MTN';
  if (airtelPrefixes.includes(prefix)) return 'AIRTEL';
  return 'UNKNOWN';
};

module.exports = { detectNetwork };
```

4.3 MTN MoMo Integration

Step 1 — Authentication (Bearer Token)

```
// Token expires every 1 hour - cache it in Redis
const getMtnToken = async () => {
  const cached = await redis.get('mtn:access_token');
  if (cached) return cached;

  const credentials = Buffer.from(
    `${process.env.MTN_API_USER}:${process.env.MTN_API_KEY}`
  ).toString('base64');

  const res = await fetch(
    'https://sandbox.momodeveloper.mtn.com/collection/token/',
    {
      method: 'POST',
      headers: {
        'Authorization': `Basic ${credentials}`,
        'Ocp-Apim-Subscription-Key': process.env.MTN_SUBSCRIPTION_KEY
      }
    }
  );
  const data = await res.json();

  // Cache for 55 minutes (token lives 60 min)
  await redis.setEx('mtn:access_token', 3300, data.access_token);
  return data.access_token;
};
```

Step 2 — Collect Payment from Customer

```
const { v4: uuidv4 } = require('uuid');

const mtnCollect = async (phoneNumber, amount, bookingId) => {
  const token = await getMtnToken();
  const referenceId = uuidv4(); // unique per transaction - store this

  await fetch(
    'https://sandbox.momodeveloper.mtn.com/collection/v1_0/requesttopay',
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'X-Reference-Id': referenceId,
        'X-Target-Environment': 'sandbox', // change to 'mtnuganda' in prod
        'Ocp-Apim-Subscription-Key': process.env.MTN_SUBSCRIPTION_KEY,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        amount: amount.toString(),
        currency: 'UGX',
        externalId: bookingId,
        payer: {
          partyIdType: 'MSISDN',
          partyId: phoneNumber // e.g. "256771234567"
        },
        payerMessage: 'Boda ride payment',
        payeeNote: `Booking #${bookingId}`
      })
    }
  );

  return referenceId; // save to payments table
};
```

Step 3 — Check Payment Status (with Polling Fallback)


```

const mtnCheckStatus = async (referenceId) => {
  const token = await getMtnToken();
  const res = await fetch(
    `https://sandbox.momodeveloper.mtn.com/collection/v1_0/requesttopay/${referenceId}`,
    {
      headers: {
        'Authorization': `Bearer ${token}`,
        'X-Target-Environment': 'sandbox',
        'Ocp-Apim-Subscription-Key': process.env.MTN_SUBSCRIPTION_KEY
      }
    }
  );
  const data = await res.json();
  return data.status; // PENDING | SUCCESSFUL | FAILED
};

// Poll every 5 seconds, up to 2 minutes
const pollMtnPayment = async (referenceId, bookingId) => {
  const maxAttempts = 24; // 24 x 5s = 2 minutes
  for (let i = 0; i < maxAttempts; i++) {
    await new Promise(r => setTimeout(r, 5000));
    const status = await mtnCheckStatus(referenceId);
    if (status === 'SUCCESSFUL') {
      await db.payments.update({ status: 'HELD', transaction_ref: referenceId })
        .where({ booking_id: bookingId });
      return 'SUCCESSFUL';
    }
    if (status === 'FAILED') return 'FAILED';
  }
  return 'TIMEOUT';
};

```

Step 4 — Disburse to Rider

```

const mtnDisburse = async (riderPhone, amount, bookingId) => {
  const token = await getMtnToken(); // use disbursement token here
  const referenceId = uuidv4();

  await fetch(
    `https://sandbox.momodeveloper.mtn.com/disbursement/v1_0/transfer`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'X-Reference-Id': referenceId,
        'X-Target-Environment': 'sandbox',
        'Ocp-Apim-Subscription-Key': process.env.MTN_DISBURSEMENT_KEY,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        amount: amount.toString(),
        currency: 'UGX',
        externalId: bookingId,
        payee: {
          partyIdType: 'MSISDN',
          partyId: riderPhone
        },
        payerMessage: 'Boda earnings',
        payeeNote: 'Trip completed'
      })
    }
  );
  return referenceId;
};

```

4.4 Airtel Money Integration

Step 1 — Authentication

```
const getAirtelToken = async () => {
  const cached = await redis.get('airtel:access_token');
  if (cached) return cached;

  const res = await fetch(
    'https://openapi.airtel.africa/auth/oauth2/token',
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        client_id: process.env.AIRTEL_CLIENT_ID,
        client_secret: process.env.AIRTEL_CLIENT_SECRET,
        grant_type: 'client_credentials'
      })
    }
  );
  const data = await res.json();
  await redis.setEx('airtel:access_token', 3300, data.access_token);
  return data.access_token;
};
```

Step 2 — Collect Payment

```
const airtelCollect = async (phoneNumber, amount, bookingId) => {
  const token = await getAirtelToken();

  const res = await fetch(
    'https://openapi.airtel.africa/merchant/v1/payments/',
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json',
        'X-Country': 'UG',
        'X-Currency': 'UGX'
      },
      body: JSON.stringify({
        reference: bookingId,
        subscriber: {
          country: 'UG',
          currency: 'UGX',
          msisdn: phoneNumber // e.g. "256751234567"
        },
        transaction: {
          amount: amount,
          country: 'UG',
          currency: 'UGX',
          id: bookingId
        }
      })
    }
  );
  return res.json();
};
```

Step 3 — Disburse to Rider

```
const airtelDisburse = async (riderPhone, amount, bookingId) => {
  const token = await getAirtelToken();
```

```

const res = await fetch(
  'https://openapi.airtel.africa/standard/v1/disbursements/',
  {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
      'X-Country': 'UG',
      'X-Currency': 'UGX'
    },
    body: JSON.stringify({
      payee: {
        msisdn: riderPhone
      },
      reference: `boda-${bookingId}`,
      pin: process.env.AIRTEL_PIN, // your merchant PIN
      transaction: {
        amount: amount,
        id: bookingId,
        type: 'B2C'
      }
    })
  }
);
return res.json();
};

```

4.5 Unified Payment Service

Wrap both MTN and Airtel into one service. The rest of your app never needs to know which network a customer is on:

```

// services/paymentService.js - single entry point for all payments

const collectPayment = async (phoneNumber, amount, bookingId) => {
  const network = detectNetwork(phoneNumber);
  if (network === 'MTN') return mtnCollect(phoneNumber, amount, bookingId);
  if (network === 'AIRTEL') return airtelCollect(phoneNumber, amount, bookingId);
  throw new Error(`Unsupported network for phone: ${phoneNumber}`);
};

const disburseToRider = async (riderPhone, amount, bookingId) => {
  const network = detectNetwork(riderPhone);
  if (network === 'MTN') return mtnDisburse(riderPhone, amount, bookingId);
  if (network === 'AIRTEL') return airtelDisburse(riderPhone, amount, bookingId);
  throw new Error(`Unsupported network for phone: ${riderPhone}`);
};

module.exports = { collectPayment, disburseToRider, detectNetwork };

```

4.6 Escrow Flow

This is the accountability mechanism for deliveries. Payment is held until the recipient confirms receipt:

```

// STEP 1: Customer books delivery - payment collected and held
const initiateDeliveryPayment = async (booking, customer) => {
  const refId = await collectPayment(customer.phone, booking.fare_estimate, booking.id);
  await db.payments.create({
    booking_id: booking.id,
    amount: booking.fare_estimate,
    method: detectNetwork(customer.phone).toLowerCase(),
    status: 'PENDING',
    transaction_ref: refId
  });
};

```

```

});
// Poll for payment confirmation
const result = await pollMtnPayment(refId, booking.id); // or Airtel equivalent
if (result === 'SUCCESSFUL') {
  await db.payments.update({ status: 'HELD' }).where({ booking_id: booking.id });
}
});

// STEP 2: Recipient confirms delivery via 6-digit OTP
const confirmDelivery = async (bookingId, enteredCode) => {
  const delivery = await db.deliveries.findOne({ booking_id: bookingId });
  if (delivery.confirmation_code !== enteredCode) throw new Error('Invalid code');

  await db.deliveries.update({ confirmed_at: new Date() }).where({ booking_id: bookingId });
  await db.payments.update({ status: 'RELEASED', released_at: new Date() })
    .where({ booking_id: bookingId });

  const booking = await db.bookings.findOne({ id: bookingId });
  const rider = await db.riders.findOne({ id: booking.rider_id });
  await disburseToRider(rider.phone, booking.fare_final * 0.85, bookingId); // 85% to rider
};

// STEP 3: Auto-release after 2 hours if unconfirmed (cron job)
const autoReleaseEscrow = async () => {
  const stalePayments = await db.payments.findAll({
    where: {
      status: 'HELD',
      held_at: { [Op.lt]: new Date(Date.now() - 2 * 60 * 60 * 1000) }
    }
  });
  for (const payment of stalePayments) {
    // Flag for admin review - do not auto-release without oversight
    await db.payments.update({ status: 'FLAGGED' }).where({ id: payment.id });
    await notifyAdmin(payment.booking_id, 'Escrow timeout - needs review');
  }
};

```

5. Real-Time GPS Tracking

5.1 Architecture

Socket.io handles bidirectional real-time communication between rider app (sending location) and customer app (receiving it). Redis stores the last known position of all online riders for fast map queries.

5.2 Server Setup

```
// server.js
const fastify = require('fastify')();
const { Server } = require('socket.io');
const { createAdapter } = require('@socket.io/redis-adapter');
const { createClient } = require('redis');

const io = new Server(fastify.server, {
  cors: { origin: '*' }
});

// Redis adapter - enables multi-server scaling later
const pubClient = createClient({ url: process.env.REDIS_URL });
const subClient = pubClient.duplicate();
await Promise.all([pubClient.connect(), subClient.connect()]);
io.adapter(createAdapter(pubClient, subClient));

io.on('connection', (socket) => {

  // Rider sends location every 3 seconds
  socket.on('rider:location', async ({ riderId, lat, lng, bookingId }) => {
    // Store in Redis (fast, ephemeral)
    await redis.hSet('riders:online', riderId, JSON.stringify({ lat, lng, updatedAt:
Date.now() }));

    // If in active booking, broadcast to customer
    if (bookingId) {
      io.to(`booking:${bookingId}`).emit('rider:moved', { lat, lng });
    }
  });

  // Customer joins booking room to receive updates
  socket.on('join:booking', ({ bookingId }) => {
    socket.join(`booking:${bookingId}`);
  });

  // Rider goes offline - remove from active map
  socket.on('disconnect', async () => {
    // In production: map socket ID to rider ID to clean up
  });
});
```

5.3 React Native — Rider Location Sender

```
// RiderApp/hooks/useLocationTracking.js
import { useEffect, useRef } from 'react';
import * as Location from 'expo-location';
```

```
import { socket } from '../services/socket';

export const useLocationTracking = (riderId, bookingId) => {
  const intervalRef = useRef(null);

  useEffect(() => {
    const startTracking = async () => {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') return;

      intervalRef.current = setInterval(async () => {
        const loc = await Location.getCurrentPositionAsync({});
        socket.emit('rider:location', {
          riderId,
          bookingId,
          lat: loc.coords.latitude,
          lng: loc.coords.longitude
        });
      }, 3000); // every 3 seconds
    };

    startTracking();
    return () => clearInterval(intervalRef.current);
  }, [riderId, bookingId]);
};
```

6. Rider Verification Flow

6.1 Onboarding Steps

```
Rider submits registration:
  → Full name + phone number
  → National ID number + photo of ID card
  → Selfie photo (face must match ID)
  → Motorcycle plate number
  → Accepts terms & code of conduct

Status: PENDING — cannot go online yet

Admin reviews in dashboard:
  → Compares selfie to ID photo
  → Checks plate number format
  → Optional: MTN KYC API cross-check on phone number

Admin decision:
  → APPROVED → status = verified → rider can go online
  → REJECTED → rider notified with reason → can resubmit

After 10 trips:
  → Community rating visible on rider profile
  → avg_rating auto-updates after every trip

Automated monitoring:
  → avg_rating < 3.5 → flagged for review
  → avg_rating < 2.5 → auto-suspended
  → 3+ cancellations in 24h → flagged
```

6.2 MTN KYC API (Optional Enhancement)

MTN provides a KYC API that lets you verify a user's identity against MTN's registered subscriber database. Use it to cross-check rider phone numbers during verification:

```
const verifyRiderKYC = async (phoneNumber, expectedName) => {
  const token = await getMtnToken();

  const res = await fetch(
    `https://sandbox.momodeveloper.mtn.com/collection/v1_0/accountholder/msisdn/${phoneNumber}/basicuserinfo`,
    {
      headers: {
        'Authorization': `Bearer ${token}`,
        'X-Target-Environment': 'sandbox',
        'Ocp-Apim-Subscription-Key': process.env.MTN_SUBSCRIPTION_KEY
      }
    }
  );
  const data = await res.json();
  // Returns: { name, given_name, family_name, status, ... }
  return {
    verified: data.status === 'ACTIVE',
    name: data.name
  };
};
```

7. Backend API Reference

7.1 Authentication Endpoints

Method	Endpoint	Body	Description
POST	/auth/send-otp	{ phone }	Send 6-digit OTP via SMS
POST	/auth/verify-otp	{ phone, otp }	Verify OTP → return JWT
POST	/auth/refresh	{ refreshToken }	Refresh access token

7.2 Booking Endpoints

Method	Endpoint	Auth	Description
POST	/bookings	Customer JWT	Create a new ride or delivery booking
GET	/bookings/:id	Any JWT	Get booking details
PATCH	/bookings/:id/accept	Rider JWT	Rider accepts a booking
PATCH	/bookings/:id/start	Rider JWT	Rider marks trip as started
PATCH	/bookings/:id/complete	Rider JWT	Rider marks trip as complete
PATCH	/bookings/:id/cancel	Any JWT	Cancel a booking
POST	/bookings/:id/rate	Customer JWT	Submit rating after trip
POST	/deliveries/:id/confirm	Recipient	Confirm delivery with OTP code

7.3 Rider Endpoints

Method	Endpoint	Auth	Description
POST	/riders/register	Public	Submit rider registration + docs
GET	/riders/nearby	Customer JWT	Get verified riders within 3km
PATCH	/riders/online	Rider JWT	Toggle rider online/offline
GET	/riders/:id/profile	Any JWT	Get rider public profile + rating
GET	/riders/me/earnings	Rider JWT	Rider earnings history

7.4 Admin Endpoints

Method	Endpoint	Auth	Description
--------	----------	------	-------------

GET	/admin/riders/pending	Admin JWT	List all unverified rider applications
PATCH	/admin/riders/:id/verify	Admin JWT	Approve or reject a rider
PATCH	/admin/riders/:id/suspend	Admin JWT	Suspend a verified rider
GET	/admin/bookings	Admin JWT	All bookings with filters
GET	/admin/payments	Admin JWT	Payment ledger + flagged escrows
POST	/admin/payments/:id/release	Admin JWT	Manually release held payment

8. Build Order & Milestones

Key Principle: Build in this order so every layer depends on something that already works. Riders must exist before customers can book them.

Phase	What to Build	Deliverable	Est. Time
1	Backend API + DB	Auth, riders, bookings, payments schema, REST endpoints	2–3 weeks
2	Admin Dashboard	React web app — rider verification, booking monitor, payment ledger	1–2 weeks
3	Rider App	React Native — registration, accept/decline trips, location tracking	2–3 weeks
4	Customer App	React Native — book ride/delivery, live tracking, payment, rating	2–3 weeks
5	MTN/Airtel Integration	Collection + disbursement + escrow + polling logic	1–2 weeks
6	Gulu Pilot	10 verified riders, 50 beta customers, real bookings	2–4 weeks

9. Environment Variables

```
# --- Server ---
PORT=3000
NODE_ENV=development
DATABASE_URL=postgresql://user:pass@localhost:5432/boda
REDIS_URL=redis://localhost:6379
JWT_SECRET=your-super-secret-key-change-in-production
JWT_REFRESH_SECRET=another-secret-key

# --- MTN MoMo ---
MTN_SUBSCRIPTION_KEY=your-mtn-subscription-key
MTN_API_USER=your-api-user-uuid
MTN_API_KEY=your-api-key
MTN_DISBURSEMENT_KEY=your-disbursement-subscription-key
MTN_TARGET_ENV=sandbox # change to mtnuganda for production

# --- Airtel Money ---
AIRTEL_CLIENT_ID=your-airtel-client-id
AIRTEL_CLIENT_SECRET=your-airtel-client-secret
AIRTEL_PIN=your-merchant-pin

# --- Africa's Talking (SMS OTP) ---
AT_API_KEY=your-africas-talking-api-key
AT_USERNAME=your-at-username

# --- Google Maps ---
GOOGLE_MAPS_API_KEY=your-google-maps-key

# --- File Storage ---
AWS_ACCESS_KEY_ID=your-aws-key
AWS_SECRET_ACCESS_KEY=your-aws-secret
AWS_S3_BUCKET=boda-rider-docs
AWS_REGION=af-south-1
```

Security: Never commit `.env` to Git. Use Railway environment variables for deployment. Rotate `JWT_SECRET` and payment keys before going to production.

10. Going Live Checklist

Before Launch

- Register a business with URSB (Uganda Registration Services Bureau)
- Apply for MTN merchant account at momodeveloper.mtn.com → Production
- Apply for Airtel merchant account at developers.airtel.africa → Production
- Register with Africa's Talking for production SMS
- Set up Google Maps API with billing — free tier covers ~28,000 map loads/month
- Deploy backend to Railway.app with production env vars
- Set up daily PostgreSQL backups
- Configure SSL certificates (Railway handles this automatically)

Rider Launch (Before Customers)

- Recruit and verify minimum 10 riders in Gulu
- Train riders on the app — acceptance rate, trip flow, how payment works
- Do 20+ test bookings internally before opening to public

Known Gotchas

MTN Token: Access tokens expire every hour. Cache in Redis and refresh proactively. Never fetch a new token on every request — you will hit rate limits.

Airtel Sandbox: Airtel sandbox sometimes returns 200 OK but the payment does not actually process. Always check the transaction status endpoint, never trust the initial response alone.

GPS in Gulu: Test Google Maps accuracy in Gulu specifically. Some northern Uganda areas have less precise map data. Consider letting riders manually confirm pickup location.

Production Agreements: Getting MTN and Airtel production access can take 2–6 weeks due to KYC and merchant agreements. Start this process early — it is the longest lead-time item.